

1)

A diferença fundamental reside em como o processo gasta o seu tempo:

- **Processos I/O-Bound (Limitados por Entrada/Saída):** São processos que passam a maior parte do tempo submetendo e aguardando solicitações de I/O (como entrada de teclado, dados de rede ou disco). Eles executam por períodos muito curtos antes de serem bloqueados à espera de mais dados.
 - Exemplos: A maioria das aplicações gráficas (GUI), editores de texto (que esperam o usuário digitar).
- **Processos CPU-Bound / Processor-Bound (Limitados pelo Processador):** São processos que passam a maior parte do tempo executando código e cálculos. Eles tendem a rodar ininterruptamente até serem preemptados pelo escalonador, pois raramente bloqueiam aguardando recursos externos.
 - Exemplos: Codificadores de vídeo, compilações de software, cálculos matemáticos intensos.

2)

O funcionamento do CFS pode ser resumido nos seguintes pontos:

- **Conceito de Justiça:** O CFS tenta modelar um processador multitarefa ideal. Nesse modelo ideal, se houvesse N processos, todos rodariam simultaneamente com $1/N$ da potência do processador. Como isso é fisicamente impossível (o processador deve rodar um processo de cada vez), o CFS aproxima esse comportamento.
- **Sem Timeslices Fixos:** Ao contrário de escalonadores Unix tradicionais, o CFS não atribui timeslices fixas baseadas na prioridade. Em vez disso, ele atribui uma proporção do tempo do processador.
- **Virtual Runtime (vruntime):** Para contabilizar o tempo, o CFS usa uma variável chamada "vruntime" (tempo de execução virtual). O "vruntime" é o tempo real de execução de um processo ponderado pelo número de processos executáveis e seus valores "nice" (prioridade).
 - Processos de alta prioridade (menor valor nice) acumulam "vruntime" mais lentamente (permitindo que rodem mais).
 - Processos de baixa prioridade acumulam "vruntime" mais rapidamente.
- **Seleção de Processo:** A lógica central do algoritmo é simples: o escalonador sempre escolhe executar o processo que possui o menor "vruntime". Isso significa que ele escolhe o processo que recebeu, até aquele momento, a menor fatia justa do processador.
- **Estrutura de Dados:** Para encontrar eficientemente o processo com o menor "vruntime", o CFS utiliza uma Árvore Rubro-Negra. Os processos são ordenados na árvore pelo valor de "vruntime". O processo a ser executado a seguir é sempre o nó mais à esquerda da árvore.

3)

- **sched_setscheduler() e sched_getscheduler():**
 - Servem para definir (set) e obter (get) a política de escalonamento de um processo (por exemplo, mudar de "SCHED_NORMAL" para uma política de tempo real como "SCHED_FIFO" ou "SCHED_RR"). O "setscheduler" também permite definir a prioridade juntamente com a política.
- **sched_setparam() e sched_getparam():**
 - Servem especificamente para definir (set) e obter (get) a prioridade de tempo real de um processo. Estas chamadas manipulam a estrutura "sched_param" que contém o valor

"rt_priority". Elas são usadas quando se quer alterar apenas a prioridade sem necessariamente mudar a política de escalonamento.